



Nombre de la práctica	Examen U4			No.	1
Asignatura:	Sistemas distribuidos	Carrera:	ISIC	Duración de la práctica (Hrs)	12

Alumnos:

Daniel Yeray Noguez Iniestra

Jesus Alberto Gonzales Sanchez

Gustavo Adrian Mendoza Monroy

Cristian Perez Hernandez

I. Competencia(s) específica(s):

Desarrollo de aplicación para alertas de emergencia de C5

II. Lugar de realización de la práctica (laboratorio, taller, aula u otro): Aula

Aula

III. Material empleado:

Computadora

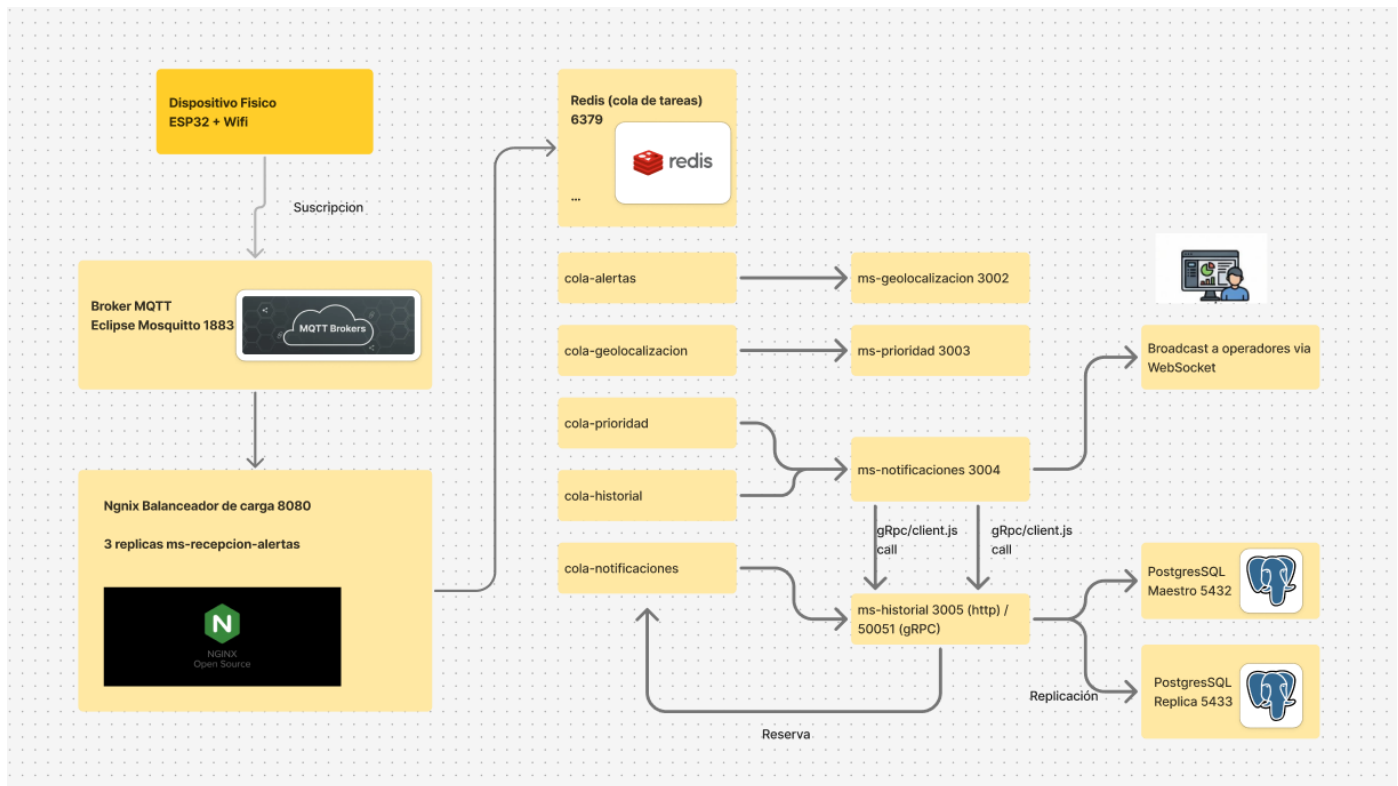
IV. Desarrollo de la práctica:

1. Resumen Ejecutivo

El Sistema C5 es una infraestructura distribuida de alerta ciudadana compuesta por **5 microservicios independientes** que procesan alertas desde dispositivos físicos (ESP32) hasta operadores en tiempo real.

2. Diagrama de Flujo del Sistema

El siguiente esquema detalla el flujo de información desde la pulsación del botón de pánico hasta la persistencia y notificación:



3. Microservicios e Infraestructura

El sistema se compone de los siguientes microservicios:

Microservicio	Puerto(s)	Función Principal	Comunicación Entrada	Comunicación Salida
ms-recepcionAlertas	3001	Recibe y valida alertas. Balanceado en 3 instancias.	MQTT (alertas)	Redis Queue (alertas_queue)
ms-geolocalizacion	3002	Geocoding inverso usando OpenStreetMap	Redis Queue (alertas_queue)	Redis Queue (geolocalizadas_queue)
ms-prioridad	3003	Clasificación por reglas (crítica, alta, media).	Redis Queue (geolocalizadas_queue)	Redis Queue (priorizadas_queue)
ms-notificaciones	3004	Broadcast a operadores y	Redis Queue (priorizadas_queue)	WebSocket y gRPC/Redis

Microservicio	Puerto(s)	Función Principal	Comunicación Entrada	Comunicación Salida
		envío a historial.		
ms-historial	3005 / 50051	Persistencia en base de datos.	gRPC / Redis Queue	PostgreSQL

Infraestructura Base

- **mqtt-broker:** Imagen eclipse-mosquitto:2 en puertos 1883/9001.
- **redis:** Imagen redis:7-alpine en puerto 6379.
- **postgres-master:** Imagen postgres:14-alpine en puerto 5432.
- **postgres-replica:** Imagen postgres:14-alpine en puerto 5433.
- **nginx-balancer:** Imagen nginx:1.21-alpine en puerto 8080.

4. Tolerancia a Fallos

- **Cola de notificaciones fallidas:** Las alertas se encolan si no hay operadores conectados y se reintentan cada 5 segundos.
- **Fallback gRPC → Redis:** Si ms-historial no está disponible, las alertas se encolan en Redis para su procesamiento posterior.
- **Alta Disponibilidad en Recepción:** MQTT reparte las alertas mediante *shared subscriptions* entre 3 instancias de recepción.
- **Tolerancia en Geolocalización:** Si OpenStreetMap tiene retrasos o falla por timeout, la alerta continúa sin bloquear el envío.
- **Replicación de BD:** El maestro replica en streaming a la réplica, asegurando que las fallas de lectura no afecten las escrituras.

5. Registros de Decisiones Arquitectónicas (ADRs)

5.1. ADR-001: Comunicación gRPC entre ms-notificaciones y ms-historial

- **Contexto:** Se requiere comunicación síncrona con confirmación de recepción y un contrato bien definido, dado que es el único punto donde se necesita el ID asignado en la base de datos.
- **Decisión:** Implementar **gRPC** del cliente al servidor, con contrato definido en *proto/historial.proto*.
- **Alternativas descartadas:** REST (HTTP/JSON) por ser más lento y carecer de contrato tipado; Redis Pub/Sub por ser "fire-and-forget".
- **Consecuencias:** Contrato verificable y tipado, confirmación explícita de DB, y mayor velocidad. Requiere mantener sincronizado el archivo *.proto* en ambos servicios.

5.2. ADR-002: Modelo de Consistencia PostgreSQL

- **Contexto:** El historial de alertas se usa para consultas y auditorías posteriores, no para toma de decisiones críticas en tiempo real (lo cual maneja WebSocket).
- **Decisión:** Adoptar **consistencia eventual**. Las escrituras van al Maestro (:5432) y las lecturas a la Réplica (:5433) vía streaming replication.
- **Alternativas descartadas:** Consistencia Fuerte (sobrecarga al maestro); Réplica Síncrona (aumenta acoplamiento y penaliza rendimiento).
- **Consecuencias:** Desacoplamiento de carga y mayor disponibilidad de lectura. Introduce un pequeño lag de replicación (aceptable para el caso de uso).

5.3. ADR-003: Redis como Bus de Mensajes

- **Contexto:** Comunicación asíncrona requerida por el stack obligatorio. No se deben perder alertas si el servicio de notificaciones se cae.
- **Decisión:** Usar Redis como **bus de mensajes basado en listas FIFO** (RPUSH / BLPOP).
- **Alternativas descartadas:** Redis Pub/Sub (viola tolerancia a fallos); Apache Kafka (sobreingeniería); RabbitMQ (componente ajeno al stack).
- **Consecuencias:** Garantiza persistencia, entrega mediante bloqueo eficiente y tolerancia a fallos.

V. Conclusiones:

El diseño arquitectónico del Sistema C5 - Alerta Ciudadana representa una solución altamente robusta, escalable y tolerante a fallos, diseñada específicamente para el manejo eficiente de emergencias en tiempo real.

A partir del análisis de su arquitectura y las decisiones tomadas (ADRs), podemos concluir lo siguiente:

- **Desacoplamiento y Resiliencia:** El uso de Redis como bus de mensajes basado en listas FIFO permite que los 5 microservicios funcionen de manera independiente. Esto asegura que, ante la caída de un componente o la ausencia de operadores, las alertas no se pierdan, sino que se encolen y se reintenten automáticamente.
- **Eficiencia en la Comunicación:** El sistema combina protocolos estratégicamente según la necesidad del flujo. Utiliza MQTT para una ingesta ligera y rápida desde los dispositivos físicos (ESP32). Emplea WebSockets para la distribución en tiempo real a los operadores. Finalmente, integra gRPC para garantizar un contrato estricto, rápido y con confirmación explícita al momento de persistir datos críticos en el historial.
- **Escalabilidad de Datos:** La implementación de PostgreSQL bajo un modelo de consistencia eventual (Maestro para escrituras, Réplica para lecturas) protege al sistema de cuellos de botella. Esto garantiza que las consultas de auditoría no penalicen la inserción masiva de nuevas alertas de emergencia.
- **Alta Disponibilidad:** Con balanceo de carga mediante Nginx en la recepción de alertas, *shared subscriptions* en MQTT y mecanismos de *fallback* (como el respaldo en Redis si gRPC falla), el sistema está diseñado para mantenerse operativo bajo estrés y recuperarse sin intervención manual tras interrupciones parciales.